Installing dependencies - Linux · macOS · Windows

> **This project runs with the language toolchains directly - there is no Docker.** You install a few tools once and run lessons with `./run -l <N>`. This guide covers **Linux, macOS, and Windows**, and all three supported developer stacks: **Python, Node.js, and C#**.

PDFs of this guide and every lesson live in `docs/pdf/` and are linked from the course site.

---

0. How it runs (no Docker) - and the polyglot model

The lessons are plain programs you run with your language's toolchain. Docker is **not** used.

This course is **polyglot** (Option B). The **Python** implementation is the reference and runs every implemented lesson today. Foundational lessons are also being implemented in **Node.js** and **C#**, selectable with a `--lang` flag:

```
./run -l 1                # Python (default / reference)
./run -l 1 --lang node    # Node.js implementation   (where available)
./run -l 1 --lang csharp  # C# / .NET implementation (where available)
```

If a language isn't ported for a lesson yet, `./run` tells you and points to the Python reference. See each lesson's **"Dependencies & Installation"** section for its language availability.

> **Node 18 and 20 are end-of-life** (April 2025 and April 2026), so every Node port now declares `engines: node >=22`. Node 24 is the current LTS if you are installing fresh.

> **Lesson 7 is the one lesson that installs a third-party package.** Lessons 1-6 need only the base requirements below; Lesson 7 rebuilds the pipeline on **LangChain**, so `./run -l 7` adds `langchain-core` and `langchain-text-splitters` (plus `@langchain/core` for the Node port) into the course virtualenv on first use. That dependency is the subject of the lesson, not an accident. Skip the install and the demo still runs the hand-rolled side and tells you the command; `./run -l 7 test` passes either way. There is no C# port - LangChain has no official .NET SDK.

---

1. Base prerequisites (every lesson)

You always need **Git**, plus the toolchain(s) for the language(s) you want to use.

Git

- **Linux (Debian/Ubuntu):** `sudo apt update && sudo apt install -y git`
- **macOS:** `brew install git` · **Windows:** `winget install -e --id Git.Git`

Get the code

```
git clone https://github.com/nikolareljin/local-ai-lab.git
cd local-ai-lab
```

Python 3.10+ (reference stack - recommended for everyone)

- **Linux (Debian/Ubuntu):** `sudo apt install -y python3 python3-venv python3-pip`
- **Linux (Fedora):** `sudo dnf install -y python3 python3-pip`
- **macOS:** `brew install python`
- **Windows:** `winget install -e --id Python.Python.3.12` (or python.org - tick **Add to PATH**)

Node.js 22+ (only for `--lang node`)

- **Linux:** NodeSource or `sudo apt install -y nodejs npm`
- **macOS:** `brew install node` · **Windows:** `winget install -e --id OpenJS.NodeJS.LTS`

.NET 8 SDK (only for `--lang csharp`, and Lesson 6)

- **Linux:** Microsoft per-distro guide
- **macOS:** `brew install dotnet@8` · **Windows:** `winget install -e --id Microsoft.DotNet.SDK.8`

Verify what you installed:

```
git --version
python3 --version    # Windows: python --version
node --version       # if using --lang node
dotnet --version     # if using --lang csharp
```

---

2. Set up the language you'll use

Python (reference)

Linux / macOS

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Windows - PowerShell

```
python -m venv venv
venv\Scripts\Activate.ps1      # if blocked: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
pip install -r requirements.txt
```

Windows - cmd

```
python -m venv venv
venv\Scripts\activate.bat
pip install -r requirements.txt
```

`requirements.txt` covers Lessons 1-2: `pypdf`, `python-docx`, `rank-bm25`, `numpy`, `flask`, `requests`, `python-dotenv`, `mcp`. (`./run -l 1` also does this automatically on first use.)

Node.js (for ported lessons)

The simplest path is `./run -l <N> --lang node`, which installs dependencies on first use. To do it by hand, run from that lesson's Node directory (e.g. `node/lesson-1/`):

```
cd node/lesson-1
npm install          # installs that lesson's package.json dependencies
```

C# / .NET (for ported lessons, and Lesson 6)

The simplest path is `./run -l <N> --lang csharp`, which restores and builds on first use. To do it by hand, run from that lesson's .NET project (e.g. `dotnet/lesson-1/`):

```
cd dotnet/lesson-1
dotnet restore       # restores NuGet packages
dotnet build -c Release
```

---

3. The default AI: Claude Code CLI (no API key)

Every stack defaults to the **Claude Code CLI** - it uses your existing Claude Code login, so there is **no API key to manage**.

Recommended - native install (self-contained, no Node.js): - **Linux / macOS** (download, inspect, then run - same caution as any remote script): `bash curl -fsSL https://claude.ai/install.sh -o claude-install.sh less claude-install.sh # review before running` After reviewing, run: `bash bash claude-install.sh` - **Windows (PowerShell)** (download, inspect, then run): `powershell iwr https://claude.ai/install.ps1 -OutFile claude-install.ps1 notepad claude-install.ps1 # review before running` After reviewing, run: `powershell .\claude-install.ps1 # if blocked: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

Alternative - via npm (if you already run Node.js 18+):

```
npm install -g @anthropic-ai/claude-code
```

Then sign in once and verify (either install):

```
claude               # first run signs you in
claude --version
```

> See the Claude Code docs for details. If `claude` isn't installed, the app says so and you can pick another provider (below).

---

4. Optional AI providers

Only needed if you set `RAG_PROVIDER` to something other than `claude`. Copy `.env.example` to `.env` for keys.

Ollama (fully local; also embeddings) - **Linux:** download the official script, **inspect it, then run** (don't pipe a remote script straight into a shell): `bash curl -fsSL https://ollama.com/install.sh -o ollama-install.sh less ollama-install.sh # review before running sh ollama-install.sh` (or follow the official Linux install docs) - **macOS:** `brew install ollama` (or download) - **Windows:** installer

```
ollama pull llama3.1           # chat / function calling
ollama pull nomic-embed-text   # embeddings (RAG_RETRIEVER=embeddings)
```

Gemini - key from aistudio.google.com → `.env`: `GEMINI_API_KEY=...` **OpenAI** - key from platform.openai.com → `.env`: `OPENAI_API_KEY=...`

---

5. Per-lesson dependencies & language availability

Lesson	Languages	Extra dependencies	Install
1 · RAG	Python ✓ · Node ◑ · C# ◑	base only (Node/C# auto-install on first `./run`)	`pip install -r requirements.txt`
2 · MCP	Python ✓ · Node (planned) · C# (planned)	`mcp` (in requirements) + Claude Code	§2 + §3
3 · LangChain	Python	LangChain + a vector store	`pip install langchain langchain-community langchain-ollama langchain-openai faiss-cpu`
4 · LangGraph	Python	LangGraph	`pip install langgraph langchain`
5 · Ollama + Function Calling	Python · Node (planned)	Ollama + a tool-capable model	§4 (Ollama) + `ollama pull llama3.1`
6 · Semantic Kernel	**C#/.NET**	.NET 8 SDK + SK NuGet	§1 (.NET) + `dotnet add package Microsoft.SemanticKernel`
7 · AWS Bedrock Agents	Python	AWS CLI + boto3	AWS CLI + `pip install boto3` + `aws configure`
8 · Google ADK	Python	`google-adk` + Gemini key	`pip install google-adk` + §4 (Gemini)

Legend: ✓ available · ◑ runnable port (BM25 retrieval; `claude`/`ollama` providers) · (planned) planned (Option B port in progress) · blank = single-language by nature.

---

6. Verify your setup (run the tests)

Each language ships an offline smoke test for Lesson 1 - no network, no API key, no LLM. They build the index over the committed sample documents and exit `0` on success. **Copy/paste and check:**

One command per language (Linux · macOS · Windows-Git-Bash)

```
./run -l 1 test                 # Python  (reference)  → "Indexed N file(s) into M chunk(s)."
./run -l 1 --lang node   test   # Node.js              → "Indexed N file(s) into M chunk(s)."
./run -l 1 --lang csharp test   # C# / .NET            → "Indexed N file(s) into M chunk(s)."
./run -l 2 test                 # Lesson 2 (MCP) tests (Python)
```

A non-zero exit means something is wrong; a printed `Indexed ...` line plus exit `0` means the stack (toolchain → extract → chunk → index) works end to end.

What "success" looks like

```
[localrag] Indexed N file(s) into M chunk(s).
```

A printed `Indexed ...` line with exit `0` is a pass. The exact counts depend on what's in `documents/`: a fresh checkout ships **two** Markdown samples, and the counts grow once you add the Caretta PDF below. (Counts also vary slightly by language for PDFs, due to text layout.) Check the exit code explicitly if you like:

```
./run -l 1 --lang node test && echo "PASS" || echo "FAIL"
```

Check the RAG behaviour (needs an AI provider, e.g. Claude Code from §3)

The best proof that RAG reads **your** file is to feed it something no model has seen. **Download and read** the short **fictional** story **The Voyage of Caretta the Magnificent** (The_Magic_Turtle_Astronaut.pdf) - a magic turtle astronaut. Read it first so you can judge the answers yourself; because it's invented, a plain LLM can't know it. Then add it to the corpus - **drop it into `documents/`, or upload it in the web UI** - and ask both a grounded question and one the story can't answer (works in any of the three languages - swap in `--lang node` / `--lang csharp`):

```
# 1) Grounded - the answer must cite [The_Magic_Turtle_Astronaut.pdf:page]
./run -l 1 ask "What was the name of Caretta's ship and where did it travel?"
#   → The ship was the Ocean's Memory [The_Magic_Turtle_Astronaut.pdf:3]; it travelled to
#     Alpha Centauri ... Sources: The_Magic_Turtle_Astronaut.pdf:3, ...

# 2) Not in the document - the anti-hallucination prompt stays honest, then labels general knowledge
./run -l 1 ask "Which dog went to space?"
#   → No dog is mentioned in your documents ... This is not covered in your documents.
#     (general knowledge - not from your documents) The most famous dog in space was Laika,
#     a Soviet dog who flew aboard Sputnik 2 in 1957 ...
```

If question 1 cites the PDF and question 2 says it's **not in your documents** before adding the clearly-labeled Laika fact, RAG + grounding is working. (See LESSON1.md → **Try it yourself** for more.)

Then launch the app

```
./run -l 1                      # Python RAG web UI   (auto-picks a free port)
./run -l 1 --lang node          # Node.js RAG web UI
./run -l 1 --lang csharp        # C# / .NET RAG web UI
```

Windows without Git Bash (PowerShell / cmd)

`./run` is a Bash script, so on Windows use **Git Bash** or **WSL** to run the commands above. If you prefer native PowerShell/cmd, call each stack directly - these are the same validation steps:

```
# Python
python -m localrag index               # → "Indexed N file(s) into M chunk(s)."
pytest -q                              # full Python test suite
python -m localrag web                 # Lesson 1 web UI
python mcp_server.py                   # Lesson 2 MCP server (stdio)

# Node.js
cd node\lesson-1; npm install; node src\cli.js index   # validate; then: node src\cli.js web

# C# / .NET
cd dotnet\lesson-1; dotnet run -c Release -- index     # validate; then: dotnet run -c Release -- web
```

---

7. Preview the docs site locally (before publishing)

The app's in-app **Troubleshooting** link normally points at the published GitHub Pages site. To review your **local** docs edits (e.g. `docs/troubleshooting.html`) before publishing, serve the `docs/` folder and point the app at it with `DOCS_BASE_URL` in `.env`:

```
# 1) Serve the local docs site (any static server works; Python is a base prereq)
python3 -m http.server 8000 --directory docs        # → http://localhost:8000/troubleshooting.html

# 2) Point the app's Troubleshooting links at your local copy (repo-root .env)
echo 'DOCS_BASE_URL=http://localhost:8000/' >> .env

# 3) Start the app - the "Troubleshooting →" link and Ollama error hints now open your local page
./run -l 1                 # or --lang node | --lang csharp
```

`DOCS_BASE_URL` is read by all three languages (Python, Node, C#) and surfaced to the web UI via `/api/status`. Remove the line (or set it back to `https://nikolareljin.github.io/local-ai-lab/`) when you're done testing.

Course: nikolareljin.github.io/local-ai-lab · Source: github.com/nikolareljin/local-ai-lab